LAB CYCLE 3
Experiment No: 5
Date: 18/03/25

Aim: Familiarization of Stored Procedure, Function, Cursor and Triggers

- **Stored Procedure**

  A stored procedure is a set of SQL commands that can be executed repeatedly with a single call.It is stored in the database and can be executed by calling its name.
  Syntax: CREATE PROCEDURE procedure_name (parameters) AS BEGIN SQL commands END;
  Example: CREATE PROCEDURE get_employee_details (emp_id INT) AS BEGIN SELECT * FROM employees WHERE emp_id = ?; END;
- **Function**

  A function is a self-contained block of code that performs a specific task.It returns a value and can be used in a SELECT statement.
  Syntax: CREATE FUNCTION function_name (parameters) RETURNS data_type AS BEGIN SQL commands RETURN result; END;
  Example: CREATE FUNCTION get_employee_name (emp_id INT) RETURNS VARCHAR(255) AS BEGIN SELECT name FROM employees WHERE emp_id = ?; END;
- **Cursor**

  A cursor is a temporary result set that allows you to process data row by row.It is used to retrieve and manipulate data in a loop.
  Syntax: DECLARE cursor_name CURSOR FOR SELECT statement;
  Example: DECLARE emp_cursor CURSOR FOR SELECT * FROM employees WHERE department = 'Sales';
- **Trigger**

  A trigger is a set of SQL commands that are executed automatically when specific event occurs (e.g., insert, update, delete).It is used to maintain data integrity and perform actions automatically.
  Syntax: CREATE TRIGGER trigger_name BEFORE/AFTER INSERT/UPDATE/DELETE ON table_name FOR EACH ROW BEGIN SQL commands END;
  Example: CREATE TRIGGER update_employee_salary BEFORE UPDATE ON employees FOR EACH ROW BEGIN IF NEW.salary > 100000 THEN SET NEW.salary = 100000; END;

1. Create a procedure which will receive account_id and amount to withdraw. If the account does not exist, it will display a message. Otherwise, if the account exists, it will allow the withdrawal only if the new balance after the withdrawal is at least 1000.

SQL:

```
delimiter //
CREATE PROCEDURE WithdrawAmount(IN account_id INT, IN
withdraw_amount DECIMAL(10,2))
BEGIN
DECLARE current_balance DECIMAL(10,2);
DECLARE account_exists INT;
SELECT COUNT(*) INTO account_exists FROM Accounts
WHERE id = account_id;
IF account_exists = 0 THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Account does not exist.';
ELSE
SELECT balance INTO current_balance FROM Accounts
WHERE id = account_id;
IF current_balance - withdraw_amount < 1000 THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Insufficient balance. The balance after withdrawal
must be at least 1000.';
ELSE
UPDATE Accounts SET balance = balance - withdraw_amount
WHERE id = account_id;
SIGNAL SQLSTATE '02000'
SET MESSAGE_TEXT = 'Withdrawal successful.';
END IF;
END IF;
END //
```

Output:

Query OK, 0 rows affected (0.07 sec)

SQL:

```
CALL WithdrawAmount(123, 500.00) ;
```

Output:

ERROR 1643 (02000): Withdrawal successful.

SQL:

```
CALL WithdrawAmount(123, 1500.00);
```

Output:

ERROR 1644 (45000): Insufficient balance. The balance after withdrawal must be at least 1000.

2. Create a ‘Customer’ table with attributes customer id, name, city and credits. Write a stored procedure to display the details of a particular customer from the customer table, where name is passed as a parameter.

SQL:

```
CREATE TABLE Customer (
        customer_id INT PRIMARY KEY AUTO_INCREMENT,
        name VARCHAR(100),
        city VARCHAR(100),
        credits DECIMAL(10,2)
    );

CREATE PROCEDURE GetCustomerDetailsByName(IN customer_name
VARCHAR(100))
    BEGIN
        SELECT * FROM Customer
        WHERE name = customer_name;
    END;
CALL GetCustomerDetailsByName('John');
```

Output:

```
+-------------+------+----------+---------+
| customer_id | name | city     | credits |
+-------------+------+----------+---------+
|           1 | John | New York | 1500.00 |
+-------------+------+----------+---------+
```

3. Create a stored procedure to determine membership of a particular customer based on the following credits:
Above 5000 = Membership Platinum
1000 to 5000 = Gold
< 1000 = silver
[Use IN and OUT Parameters]

SQL:

```
CREATE PROCEDURE DetermineMembership(IN customer_credits DECIMAL(10,2),
OUT membership_level VARCHAR(50))
    BEGIN
    IF customer_credits >= 5000 THEN
        SET membership_level = 'Membership Platinum';
        ELSEIF customer_credits >= 1000 THEN
        SET membership_level = 'Gold';
      ELSE
        SET membership_level = 'Silver';
      END IF;
```

```
CALL DetermineMembership(6000.00, @membership1);
CALL DetermineMembership(3500.00, @membership2);
CALL DetermineMembership(800.00, @membership3);
CALL DetermineMembership(10000.00, @membership4);

SELECT @membership1 AS John_Membership, @membership2 AS
Alice_Membership, @membership3 AS Bob_Membership, @membership4 AS Emma;
```

Output:

```
+---------------------+------------------+----------------+---------------------+
| John_Membership     | Alice_Membership | Bob_Membershi |Emma    |
+---------------------+------------------+----------------+---------------------+
| Membership Platinum | Gold             | Silver         | Membership Platinum |
+---------------------+------------------+----------------+---------------------+
```

4. Write a function that takes employee name as parameter and returns the number of employees with this name. Use the function to update details of employees with unique names. For other cases, the program (not the function) should display error messages - "No Employee" or "Multiple employees".

SQL:

```
CREATE FUNCTION CountEmployeesWithName(employee_name VARCHAR(100))
RETURNS INT
      DETERMINISTIC
      BEGIN
        DECLARE employee_count INT;

        SELECT COUNT(*) INTO employee_count FROM Employees
         WHERE name = employee_name;

        RETURN employee_count;
    END ;

SELECT CountEmployeesWithName('John');
```

Output:

```
+---------------------------------+
| CountEmployeesWithName('John') |
+---------------------------------+
|                               2 |
+---------------------------------+
```

5. Write a stored procedure using cursor to calculate salary of each employee. Consider an Emp_salary table have the following attributes emp_id, emp_name, no_of_working_days, designation and salary.

SQL:

```
CREATE PROCEDURE CalculateEmployeeSalary()
   -> BEGIN
   ->     DECLARE done INT DEFAULT FALSE;
   ->     DECLARE emp_id_val INT;
   ->     DECLARE emp_name_val VARCHAR(100);
   ->     DECLARE working_days_val INT;
   ->     DECLARE designation_val VARCHAR(100);
   ->     DECLARE daily_wage DECIMAL(10, 2);
   ->     DECLARE emp_salary DECIMAL(10, 2);
   ->
   ->     DECLARE emp_cursor CURSOR FOR
   ->         SELECT emp_id, emp_name, no_of_working_days, designation
   ->         FROM Emp_salary;
   ->
   ->     DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;
   ->
   ->     OPEN emp_cursor;
   ->
   ->     emp_loop: LOOP
   ->         FETCH emp_cursor INTO emp_id_val, emp_name_val, working_days_val,
designation_val;
   ->
   ->         IF done THEN
   ->             LEAVE emp_loop;
   ->         END IF;
   ->
   ->         CASE designation_val
   ->             WHEN 'Assistant Professor' THEN SET daily_wage := 1750.00;
   ->             WHEN 'Clerk' THEN SET daily_wage := 750.00;
   ->             WHEN 'Programmer' THEN SET daily_wage := 1250.00;
   ->             ELSE SET daily_wage := 0.00;
   ->         END CASE;
   ->
   ->         SET emp_salary := daily_wage * working_days_val;
   ->
   ->         SELECT CONCAT(emp_name_val, "'s salary is $", emp_salary) AS Result;
   ->
   ->     END LOOP;
   ->
   ->     CLOSE emp_cursor;
   -> END ;

CALL CalculateEmployeeSalary();
```

Output:

```
+----------------------------+
| Result                     |
+----------------------------+
| John's salary is $35000.00 |
+----------------------------+

+-----------------------------+
| Result                      |
+-----------------------------+
| Alice's salary is $18750.00 |
+-----------------------------+
+---------------------------+
| Result                    |
+---------------------------+
| Bob's salary is $27500.00 |
+---------------------------+
```

6. Write a procedure to calculate the electricity bill of all customers. Electricity board charges the following rates to domestic uses to find the consumption of energy.

a) For first 100 units Rs:2 per unit.
b) 101 to 200 units Rs:2.5 per unit.
c) 201 to 300 units Rs: 3 per unit.
d) Above 300 units Rs: 4 per unit

Consider the table 'Bill' with fields customer_id, name, pre_reading, cur_reading , unit, and amount.

SQL:

```
CREATE PROCEDURE CalculateElectricityBill()
   -> BEGIN
   ->    DECLARE customer_id_val INT;
   ->    DECLARE name_val VARCHAR(100);
   ->    DECLARE pre_reading_val INT;
   ->    DECLARE cur_reading_val INT;
   ->    DECLARE unit_val INT;
   ->    DECLARE amount_val DECIMAL(10, 2);
   ->
   ->    DECLARE done INT DEFAULT FALSE;
   ->
   ->    DECLARE bill_cursor CURSOR FOR
   ->       SELECT customer_id, name, pre_reading, cur_reading
   ->       FROM Bill;
   ->
   ->    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;
   ->
   ->    OPEN bill_cursor;
   ->
```

```
    ->    bill_loop: LOOP
    ->        FETCH bill_cursor INTO customer_id_val, name_val, pre_reading_val,
cur_reading_val;
    ->
    ->        IF done THEN
    ->            LEAVE bill_loop;
    ->        END IF;
    ->
    ->        SET unit_val := cur_reading_val - pre_reading_val;
    ->
    ->        IF unit_val <= 100 THEN
    ->            SET amount_val := unit_val * 2.00;
    ->        ELSEIF unit_val <= 200 THEN
    ->            SET amount_val := 100 * 2.00 + (unit_val - 100) * 2.50;
    ->        ELSEIF unit_val <= 300 THEN
    ->            SET amount_val := 100 * 2.00 + 100 * 2.50 + (unit_val - 200) * 3.00;
    ->        ELSE
    ->            SET amount_val := 100 * 2.00 + 100 * 2.50 + 100 * 3.00 + (unit_val - 300)
* 4.00;
    ->        END IF;
    ->
    ->        -- Update the amount in the Bill table
    ->        UPDATE Bill
    ->        SET amount = amount_val
    ->        WHERE customer_id = customer_id_val;
    ->    END LOOP;
    ->
    ->    CLOSE bill_cursor;
    -> END ;
CALL CalculateElectricityBill();
```

Output:

customer_id	name	pre_reading	cur_reading	unit	amount
1	John	100	200	NULL	200.00
2	Alice	150	300	NULL	325.00
3	Bob	200	400	NULL	450.00
4	Emma	250	500	NULL	600.00
5	Sophia	300	450	NULL	325.00

7. Create a trigger on employee table such that whenever a row is deleted, it is moved to history table named ‘Emp_history’ with the same structure as employee table. ‘Emp_history’ will contain an additional column “Date_of_deletion” to store the date on which the row is removed. [After Delete Trigger]

SQL:

```
CREATE TRIGGER MoveDeletedEmployeeToHistory
   -> AFTER DELETE ON Employee
   -> FOR EACH ROW
   -> BEGIN
   ->     INSERT INTO Emp_history (employee_id, name, department, salary)
   ->     VALUES (OLD.employee_id, OLD.name, OLD.department, OLD.salary);
   -> END ;
```

select * from Emp_history

Output:

```
+-------------+-------+------------+----------+---------------------+
| employee_id | name  | department | salary   | date_of_deletion    |
+-------------+-------+------------+----------+---------------------+
|           2 | Alice | HR         | 45000.00 | 2024-06-03 01:11:43 |
+-------------+-------+------------+----------+---------------------
```

8. Before insert a new record in emp_details table, create a trigger that check the column value of FIRST_NAME, LAST_NAME, JOB_ID and if there are any space(s) before or after the FIRST_NAME, LAST_NAME, TRIM () function will remove those. The value of the JOB_ID will be converted to upper cases by UPPER () function. [Before Insert Trigger]

SQL:

```
CREATE TRIGGER BeforeInsertEmpDetails
   -> BEFORE INSERT ON emp_details
   -> FOR EACH ROW
   -> BEGIN
   ->     -- Trim leading and trailing spaces from FIRST_NAME and LAST_NAME
   ->     SET NEW.FIRST_NAME = TRIM(NEW.FIRST_NAME);
   ->     SET NEW.LAST_NAME = TRIM(NEW.LAST_NAME);
   ->
   ->     -- Convert JOB_ID to upper case
   ->     SET NEW.JOB_ID = UPPER(NEW.JOB_ID);
   -> END ;
```

INSERT INTO emp_details (FIRST_NAME, LAST_NAME, JOB_ID) VALUES (' Michael ', ' Smith ', 'analyst');

SELECT * FROM emp_details;

Output:

```
+--------+------------+-------------+----------+
| emp_id | FIRST_NAME | LAST_NAME   | JOB_ID   |
+--------+------------+-------------+----------+
|      1 |    John    |    Doe      | engineer |
|      2 | Alice      | Smith       | clerk    |
|      3 |   Bob      |   Johnson   | manager  |
|      4 | Michael    | Smith       | ANALYST  |
+--------+------------+-------------+----------+
```

9. Consider the following table with sample data. Create a trigger to calculate total marks, percentage and grade of the students, when marks of the subjects are updated. [After Update Trigger]

For this sample calculation, the following conditions are assumed: Total Marks (will be stored in TOTAL column) : TOTAL = SUB1 + SUB2 + SUB3 + SUB4 + SUB5. Percentage of Marks (will be stored in PER_MARKS column): PER_MARKS = (TOTAL)/5 Grade (will be stored in GRADE column):
- If PER_MARKS>=90 -> 'EXCELLENT'
- If PER_MARKS>=75 AND PER_MARKS 'VERY GOOD'
- If PER_MARKS>=60 AND PER_MARKS 'GOOD'
- If PER_MARKS>=40 AND PER_MARKS 'AVERAGE'
- If PER_MARKS 'NOT PROMOTED

STUDENT_ID	NAME	SUB1	SUB2	SUB3	SUB4	SUB5	TOTAL	PER_MARKS	GRADE
1	Steven King	0	0	0	0	0	0	0.00	
2	Neena Kochhar	0	0	0	0	0	0	0.00	
3	Lex De Haan	0	0	0	0	0	0	0.00	
4	Alexander Hunold	0	0	0	0	0	0	0.00	

SQL:

```
CREATE TRIGGER CalculateStudentDetails
 -> BEFORE UPDATE ON student_marks
 -> FOR EACH ROW
 -> BEGIN
 ->     -- Calculate total marks
 ->     SET NEW.total = NEW.sub1 + NEW.sub2 + NEW.sub3 + NEW.sub4 +
NEW.sub5;
 ->
 ->     -- Calculate percentage of marks
 ->     SET NEW.per_marks = NEW.total / 5;
 ->
 ->     -- Determine grade based on percentage
 ->     IF NEW.per_marks >= 90 THEN
 ->         SET NEW.grade = 'EXCELLENT';
 ->     ELSEIF NEW.per_marks >= 75 THEN
 ->         SET NEW.grade = 'VERY GOOD';
 ->     ELSEIF NEW.per_marks >= 60 THEN
 ->         SET NEW.grade = 'GOOD';
 ->     ELSEIF NEW.per_marks >= 40 THEN
 ->         SET NEW.grade = 'AVERAGE';
```

```
    ->    ELSE
    ->        SET NEW.grade = 'NOT PROMOTED';
    ->    END IF;
    -> END ;
UPDATE student_marks
    -> SET sub1 = 80, sub2 = 85, sub3 = 90, sub4 = 75, sub5 = 85
    -> WHERE student_id = 1;
select * from student_marks;
```

Output:

```
+------------+-------+------+------+------+------+------+-------+-----------+-----------+
| student_id | name  | sub1 | sub2 | sub3 | sub4 | sub5 | total | per_marks | grade     |
+------------+-------+------+------+------+------+------+-------+-----------+-----------+
|          1 | John  |   80 |   85 |   90 |   75 |   85 |   415 |     83.00 | VERY GOOD |
|          2 | Alice |   70 |   80 |   65 |   75 |   85 |  NULL |      NULL | NULL      |
|          3 | Bob   |   60 |   65 |   70 |   55 |   80 |  NULL |      NULL | NULL      |
|          4 | Emma  |   90 |   85 |   95 |   88 |   92 |  NULL |      NULL | NULL      |
+------------+-------+------+------+------+------+------+-------+-----------+-----------+
```

Result:

Query is executed successfully and has been executed